Atividade de Programação[1]
Interpretador LPAS

Terça-feira, 7 de outubro de 2025.

O processador executa as instruções de um programa em uma série de pequenos passos:

1. Busca a próxima instrução da memória para o registrador de instrução.
2. Atualiza o contador de programa para indicar a instrução seguinte.
3. Determina o tipo da instrução.
4. Se a instrução usa dados da memória, determina onde eles estão.
5. Busca os dados, se houver algum, para registradores internos da CPU.
6. Executa a instrução.
7. Volta ao passo 1 para iniciar a execução da próxima instrução.

Essa sequência de passos é frequentemente chamada de **ciclo busca-decodifica-executa**. Ela é o centro da operação de todos os computadores. Pode-se construir um programa capaz de executar esse ciclo para executar as instruções de outro programa. O programa capaz de realizar esse ciclo é chamado de **interpretador**. A figura abaixo apresenta um exemplo de um interpretador[2] para um computador simples.

```
typedef unsigned int Palavra; // Palavra de 32 bits
typedef unsigned int Endereco; // Endereço de 32 bits

void interpretador(Palavra memoria[], Endereco enderecoInicial, unsigned short tamanhoMemoria) {
    Endereco enderecoDado;
    Palavra instrucao;
    int execucao = 1;

    // Registrador que possui o endereço da próxima instrução.
    Endereco pc = enderecoInicial;
    while (execucao)
    {
        instrucao = memoria[pc]; // Passo 1
        pc = pc + 1; // Passo 2
        int codigoInstrucao = identificarInstrucao(instrucao); // Passo 3
        enderecoDado = localizarDado(instrucao, memoria, tamanhoMemoria); // Passo 4

        // Verifica se a instrução possui operando.
        if (enderecoDado >= 0) dado = memoria[enderecoDado]; // Passo 5
        execucao = executarInstrucao(codigoInstrucao, dado); // Passo 6
    }
}
```

[1] Atualizado em 13/01/2026.
[2] Adaptado de TANENBAUM, A. S.; AUSTIN, T. **Organização Estruturada de Computadores**. 6ª edição. São Paulo: Pearson Prentice Hall, 2013.

Use o algoritmo da figura acima para desenvolver um interpretador LPAS - Linguagem de Programação para Aritmética Simples. Essa linguagem de máquina (ver Tabela 1) é usada para escrever programas que executam as operações aritméticas somente com números inteiros. LPAS possui a seguinte organização:

1. Cada linha de código deve ter apenas uma única instrução.
2. As instruções e os seus argumentos variáveis devem ser escritos em maiúsculo.
3. Os comentários devem iniciar com ponto-e-vírgula.
4. Os programas LPAS são executados por um interpretador, que é um programa que lê o código-fonte LPAS e executa suas instruções uma a uma. Um programa é executado por vez.
5. O interpretador é responsável por carregar o programa para a memória e executá-lo.
6. O interpretador possui duas memórias (código e dados) e um único registrador que permite armazenar um número inteiro.
7. A memória de código armazena os programas LPAS a serem executados pelo interpretador.
8. A memória de dados armazena as variáveis do programa que está em execução. Todas as variáveis são do tipo de dado inteiro.

O interpretador dever ser escrito para o sistema operacional Linux usando a Linguagem C padrão ISO, a GNU C Library (glibc) e a IDE *Visual Studio Code*.

Exemplos de programas LPAS:

```
; Exibe os números de 1 a 5.

WRITE 1
WRITE 2
WRITE 3
WRITE 4
WRITE 5
HALT

; Lê e exibe um número.

READ X
WRITE X
HALT

; Executa a soma de 2 números inteiros.

READ X      ; X recebe o valor lido do teclado
READ Y      ; Y recebe o valor lido do teclado
LOAD X      ; carrega o valor de X no registrador (registrador ⇦ X)
ADD Y       ; soma Y com X e coloca o resultado no registrador (registrador ⇦ registrador + Y)
STORE Z     ; armazena o resultado da soma na variável Z (Z ⇦ registrador)
WRITE Z     ; escreve no vídeo o valor de Z (o resultado da soma)
HALT        ; finaliza o programa
```

Prof. Márlon Oliveira da Silva
marlon.silva@ifsudestemg.edu.br

Tabela 1: Instruções de máquina LPAS

Código	Instrução	Descrição
10	READ	Lê um valor do teclado para a variável. Ex.: **READ X** ; lê um valor do teclado e armazena na variável X
11	WRITE	Escreve no vídeo o valor da variável ou número. Ex.: **WRITE X** ; escreve no vídeo o valor da variável X **WRITE 10** ; escreve no vídeo o número 10
21	LOAD	Carrega o valor da variável para o registrador. Ex.: **LOAD X** ; registrador ⇦ X
22	STORE	Armazena o valor do registrador em uma variável. Ex.: **STORE X** ; X ⇦ registrador
30	ADD	Adiciona o valor do registrador ao valor da variável ou número e armazena o resultado no registrador. Ex.: **ADD X** ; registrador ⇦ registrador + X **ADD 20** ; registrador ⇦ registrador + 20
31	SUB	Subtrai o valor do registrador do valor da variável ou número e armazena o resultado no registrador. Ex.: **SUB X** ; registrador ⇦ registrador - X **SUB 20** ; registrador ⇦ registrador - 20
32	MUL	Multiplica o valor do registrador com o valor da variável ou número e armazena o resultado no registrador. Ex.: **MUL X** ; registrador ⇦ registrador * X **MUL 20** ; registrador ⇦ registrador * 20
33	DIV	Divide o valor do registrador pelo valor da variável ou número e armazena o resultado no registrador. Ex.: **DIV X** ; registrador ⇦ registrador / X **DIV 20** ; registrador ⇦ registrador / 20
50	HALT	Finaliza o programa.